

Linguaggio C

Ripasso

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio nonché di Sistemi Operativi del Prof. Bini.

Le origini del C

- C si è evoluto da due linguaggi precedenti: BCPL e B.
 - 1. BCPL è stato sviluppato nel 1967 da Martin Richards come linguaggio per la scrittura di sistemi operativi e compilatori.
 - 2. Ken Thompson ha modellato molte caratteristiche del suo linguaggio B sulla base delle loro controparti in BCPL e, nel 1970, ha utilizzato B per creare le prime versioni del sistema operativo UNIX presso i Bell Laboratories.
- C è stato sviluppato a partire da B da Dennis Ritchie ai Bell Laboratories
 - È stato implementato per la prima volta nel 1972.
- C inizialmente divenne ampiamente noto come il linguaggio di sviluppo di UNIX
 - Ancora oggi molti OS sono scritti in C o C++

Standardizzazione del C

- La rapida espansione di C su vari hardware ha portato a molte versioni di C
 - Versioni simili ma spesso incompatibili
 - Problema per programmatori che volevano sviluppare codice per diverse piattaforme
- Primo standard approvato nel 1989, negli Stati Uniti dall'American National Standards Institute (ANSI)
 - Questo C è noto come ANSI C
- ... poi in tutto il mondo attraverso l'International Standards Organization (ISO):
 - Standard attuale: ISO/IEC 9899:2024 detto C23 e pubblicato a fine 2024

Imparare il C

- I programmi in C sono costituiti da pezzi chiamati funzioni
 - È possibile programmare tutte le funzioni necessarie per formare un programma C
 - Tuttavia, i programmatori in C sfruttano la ricca raccolta di funzioni disponibili nella libreria standard C.
- Tre cose almeno sono da considerare per imparare a programmare in C:
 1. Imparare i costrutti e i principi di base della “programmazione imperativa procedurale” e delle strutture dati
 2. Imparare il linguaggio C (sintassi e semantica)
 3. Imparare a usare le funzioni nella libreria standard C

Librerie C Standard e Open-Source

- Programmando in C in genere utilizzerete i seguenti elementi costitutivi:
 1. funzioni di libreria standard C
 2. funzioni di librerie C open source
 3. funzioni che creerete voi stessi

} **Software Reuse**
- Software reuse:
 - Incrementa la vostra produttività
- Librerie Open Source:
 - [Github](#) elenca oltre 32000 progetti in C:
 - [Awesome C](#) fornisce elenchi curati di librerie C popolari

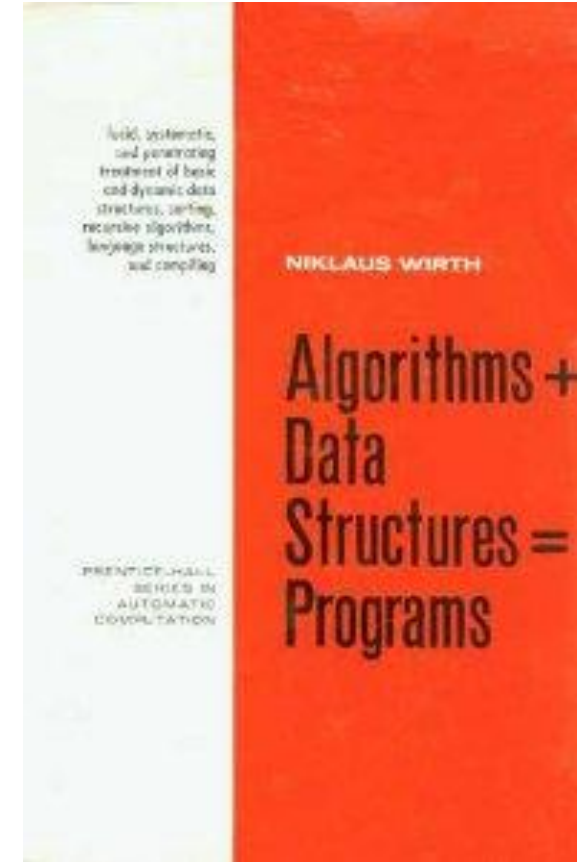
Cos'è un Programma?

"Algorithms + Data Structures = Programs"

– Nicholas Wirth

- O in maniera più esplicativa:
 - **Procedimento algoritmico** applicato ad un problema dato da automatizzare, tipicamente **codificato** in una serie di linee di codice, (**istruzioni scritte in un linguaggio di programmazione**), che può essere **eseguito** o interpretato da un elaboratore, **ricevendo** in input determinati **dati e restituendo** in output eventuali **risultati** ottenuti a seguito dell'esecuzione delle sue istruzioni

Linguaggio C



Programma in Linguaggio C I

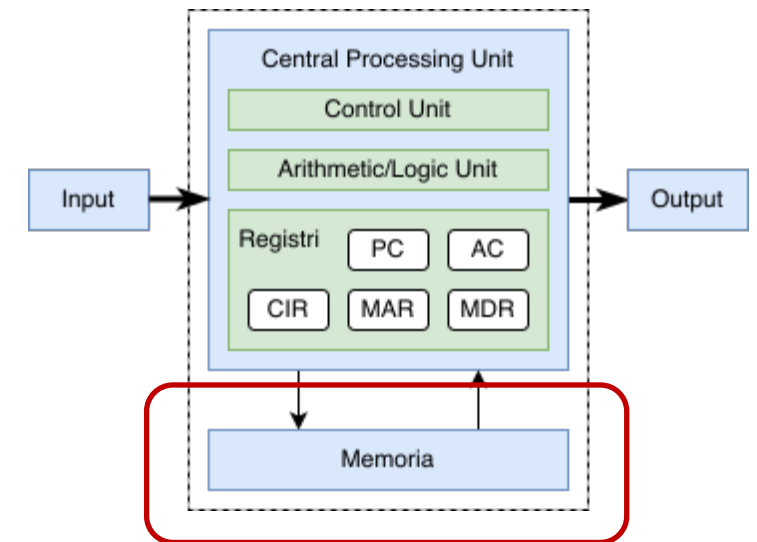
- **Linguaggio C**
 - Linguaggio di programmazione implementa i **paradigma**
 - **imperativo**
 - Ogni istruzione è un comando per il computer
 - Altri paradigmi: dichiarativo, funzionale
 - **procedurale**
 - Il programma è diviso in blocchi di codice ben delimitati detti sottoprogrammi, procedure o **funzioni** in base al linguaggio e ruolo
 - In C solo funzioni

Programma in Linguaggio C II

- **C** è costruito per l'efficienza
 - Ampiamente usato per sviluppare sistemi che richiedono efficienza
 - Risparmio di memoria, velocità, scalabilità, ...
 - Esempi: sistemi operativi (tra cui i kernel di Android e di iOS), embedded, real-time, ...
- In C:
 - Concetto base: **variabile**
 - Operazione base: **assegnamento** a variabile
 - Programma: **sequenza strutturata** di istruzioni che operano su variabili

Cos'è una variabile in C? I

- C ha un approccio vicino all'implementazione
 - Tiene conto di come le operazioni vengono eseguite sulla macchina
- C vede una variabile come una porzione di memoria
 - In un certo senso le variabili non esistono, esiste solo la memoria
 - Le variabili sono un metodo comodo per riferirci a porzioni di memoria



Cos'è una Variabile in C? II

- In C ci interessa solo:
 - **dove** in memoria è allocata una variabile
 - **quanto** spazio di memoria che occupa
 - il **contenuto**: i bytes in memoria
- In questa visione, il C ha due operatori:
 - "Dammi l'indirizzo di memoria di questa variabile"
 - "Dammi il contenuto della memoria all'indirizzo di questa variabile»"
- Il tipo della variabile specifica come interpretare i bytes in memoria

Variabili in C

- Cos'è una variabile in C?
 - C ha un approccio vicino all'implementazione
 - Tiene conto di come le operazioni vengono eseguite sulla macchina
 - C vede una variabile come una porzione di memoria
 - In un certo senso le variabili non esistono, esiste solo la memoria
 - Le variabili sono un metodo comodo per riferirci a porzioni di memoria
- In C ci interessa solo:
 - **dove** in memoria è allocata una variabile
 - **quanto** spazio di memoria che occupa
 - il **contenuto**: i bytes in memoria

La Memoria in C I

- In C la memoria:
 - È vista come una lunghissima sequenza di byte
 - È indirizzabile al byte
 - Ogni indirizzo contiene un solo byte
- Una variabile
 - Occupa uno o più byte
 - asseconda del tipo

indirizzo	contenuto
...	...
7FFF050E785AB301	B5
7FFF050E785AB302	C3
7FFF050E785AB303	AA
7FFF050E785AB304	00
7FFF050E785AB305	01
...	...

La Memoria in C II

- La memoria può essere vista
 - Come una lunghissima strada
 - Ogni variabile è una persona che abita in una casa identificata dal nome
 - Ogni casa ha un numero civico (indirizzo)
 - Ogni casa è di un tipo standard
 - Occupa più o meno spazio
 - Garage → char
 - Casetta → int
 - Villa → double
 - Condominio → array



SDB_0.1 "A long street with several houses"

Uso delle Variabili in C I

- Dichiarazione di una variabile
 - Informa il compilatore del tipo e conseguentemente di quanto memoria richiede la variabile
 - Esempio:
 - `int a` dichiara una variabile di tipo intero
- L'identificatore è il nome della variabile
 - Può contenere caratteri alfanumerici più il carattere "_"
 - Non può iniziare con un numero
 - Non può essere una parola chiave riservata del linguaggio (for, while, ...)
 - Nel nostro esempio: `a`

Uso delle Variabili in C II

- Opzionalmente, una variabile può essere inizializzata con una costante:

`int a;`  `/* non inizializzata */`

`int b = 91;`  `/* inizializzata */`

`int c = my_function();`  `/* errore: valore non costante */`

- Una variabile è una porzione di memoria
 - Non è mai vuota: contiene il valore dei byte in memoria
 - Non assumete che il valore iniziale sia zero (o altro)
 - Inizializzate sempre le variabili**

Tipi di Variabili (ANSI C89)

- Tipi di variabili in C sono:
 - Interi di grandezza crescente: `(char)`, `(short)`, `(int)`, `(long)`
 - Numeri con la virgola: `(float)`, `(double)`
 - Puntatori (cioè, indirizzi di memoria): `(char *)`, `(int *)`, `(double *)`, `(void *)`
 - Ultimi standard ISO stanno aggiungendo altri tipi
 - Esempio: C24 aggiunge `bool`
- Lo spazio in byte dei tipi dipende dalla piattaforma hardware
 - L'operatore `sizeof()` permette di conoscere la dimensione in byte
 - `sizeof(int)` dimensione in bytes di una variabile di tipo `int`
 - `sizeof(a)` dimensione in bytes della variabile `a`

Uso della Tipizzazione nei Linguaggi Di Programmazione (LP)

- Tipizzazione statica vs. dinamica:
 - Statica se il tipo delle variabile e espressioni è conosciuto al tempo della compilazione
 - Dinamica se a tempo di esecuzione il tipo associato a ciascuna variabile/espressione è noto quando viene accesso/valutato
- Type-safe:
 - Un LP è type-safe se le uniche operazioni che possono essere eseguite sui dati sono quelle autorizzate dal tipo di dato
 - Qualsiasi tentativo di usare un valore in modo errato rispetto al suo tipo è garantito di causare un errore (sia a tempo di compilazione che a tempo di esecuzione)
- Type-sound:
 - Un LP tipizzato staticamente è type-sound se "programmi ben tipizzati non possono 'andare storti'"
 - type-sound(ness) implica type-safe(ty)

C usa
tipizzazione statica

C non è
type-safe

C non è
type-sound

Perché Importante? Il Caso Arienne 5 nel 1996

- Il lancio 501 di Arienne 5 si è concluso con un fallimento a causa di molteplici errori nella progettazione del software.
- Il software di controllo era scritto in Ada
 - Ada genera un'eccezione in caso di overflow/underflow in un'operazione aritmetica
- I motori del nuovo razzo Arienne 5 erano più potenti di Arienne 4
 - L'accelerazione più grande ha causato un overflow nel calcolo dell'accelerazione istantanea (salvata in una variabile di tipo intero)
 - Il codice non gestiva in maniera corretta quest'eccezione
 - Questo ha portato all'arresto l'intero sistema di navigazione inerziale.
 - Ciò ha fatto deviare il razzo, che ha iniziato a disintegrarsi a causa delle forze aerodinamiche, e infine si è autodistrutto tramite il suo sistema di terminazione del volo automatizzato.
- Il fallimento è noto come uno dei bug software più costosi della storia

Alcune Insidie del C: Errori da Evitare

- **if-statement**

- L'espressione che esprime la condizione in un if-statement può essere tante cose
 - Esempi: int, long, char, puntatore, ...
- La condizione è considerata vera se il valore dell'espressione è diverso da 0, falsa altrimenti.
- Attenti all'uso di = vs ==

```
{  
    ...  
    if (x == 5) {  
        z++;  
    }  
    ...  
}
```

```
{  
    ...  
    if (x = 5) {  
        z++;  
    }  
    ...  
}
```

L'espressione
assegna 5 a x
e esegue l'if

Alcune Insidie del C: Errori da Evitare

- **Overflow/underflow**

- Se durante la valutazione di un'espressione di tipo intero avviene un overflow/underflow l'esecuzione prosegue, ma il risultato del calcolo sarà di norma sbagliato.

Quanti errori?

Non è l'unica
soluzione possibile

```
#include <stdio.h>
```

```
int main(void)
```

```
{  
    char a = 128;  
    char b = 128;  
    char s = 0;
```

```
  
    s = a + b;  
    printf("%d + %d = %d\n", a, b, s);  
}
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{  
    unsigned char a = 128;  
    unsigned char b = 128;  
    short s = 0;
```

```
  
    s = a + b;  
    printf("%d + %d = %d\n", a, b, s);  
}
```

Alcune Insidie del C: Errori da Evitare

- C non fa controlli sull'accesso in memoria
- Caso Array:
 - array `a` di elementi di tipo `T` dichiarata come `T []` o `T [n]` lunghezza $n \geq 1$,
 - memorizzato a partire dall'indirizzo di memoria `m`
- e la variabile `i` contiene un valore $p \geq n$
 - allora `a[i]` accede ad un indirizzo di memoria non valido
 - stessa cosa con `*(a+i)`
 - `a+i` è calcolato in C come
$$m + (p * \text{sizeof}(T))$$

```
#include <stdio.h>

int main(void)
{
    ssize_t i = 3;
    char a[] = {1, 2, 3, 4, 5, 6};

    printf("a ha %lu elementi\n", sizeof(a) / sizeof(char));

    printf("a[%zd] = %d\n", i, a[i]);
    printf("*(a+%zd) = %d\n", i, *(a + i));

    i = 10;
    printf("a[%zd] = %d\n", i, a[i]);

    i = -5;
    printf("a[%zd] = %d\n", i, a[i]);
}
```

Alcune Insidie del C: Errori da Evitare

- C non fa controlli sull'accesso in memoria
- Caso memoria dinamica:

```
ptr = malloc(n * sizeof(T));  
free(ptr);
```

 - `free()` libera la memoria ma non cambia il valore di `ptr`
 - Usare `ptr` dopo `free` può puntare a memoria «casuale» che cambia ad ogni esecuzione
- In entrambi i casi il programma avrà comportamenti strani difficili da debuggare.

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main(void)  
{  
    char *ptr = NULL;  
    int n = 6;  
  
    printf("L'indirizzo di ptr è %p\n", ptr);  
  
    ptr = (char *)malloc(n * sizeof(char));  
    printf("L'indirizzo di ptr è %p\n", ptr);  
  
    free(ptr);  
    printf("L'indirizzo di ptr è %p\n", ptr);  
}
```